

One Pager – Spring Application Events

Wat is een applicatie event

Dit is een manier waarop je je applicatie kan opbouwen en laten communiceren via events. Zo kan je bijvoorbeeld een component in je applicatie, zoals een service, een event publiceren. Erna kunnen één of meerdere plaatsen in je applicatie deze worden opgevangen door “@EventListener” boven je methode te doen. Je kan bij deze dan ook Keys definiëren. Wanneer het event wordt opgevangen kan je daar specifieke code uitvoeren. Dit zal zorgen voor wat we noemen een loose coupling, dit betekent dat je applicatie heel onafhankelijk van elkaar is tussen componenten.

Waar kunnen wij dit in ons project gebruiken

Je zou dit op veel manieren kunnen implementeren, maar in ons geval is er een geval dat heel duidelijk is hoe het zou kunnen helpen. Dit is in ons geval de events te maken met order maken tot order Delivery. Hier gebruiken we RabbitMQ om met meerdere service met elkaar te praten. En bij sommige van deze events moeten we dan ook meerdere RabbitMQ message sturen. Nu gebeuren deze altijd via aparte methodes, maar via event listener kunnen we 1 event publiceren en dan op meerdere methode via de @EventListener deze opvangen en dan via hier de messages sturen.

Voordelen en Nadelen

We gaan nu overlopen welke voordelen en nadelen het gebruiken Application Events. Het eerste voordeel is al eerdere aangehaald met de Loose coupling dus we gaan meteen naar de tweede gaan. Deze is Scalability: Als je nieuwe functionaliteiten wilt toevoegen aan de applicatie moet je niet helemaal in je service gaan en allemaal dingen gaan aanpassen. Je kan gewoon een extra event listener maken en hier dan de nieuwe functionaliteit uitwerken.

Dan de volgende die ik ga aanhalen is de Asynchroon werking die je kan doen: Je kan heel gemakkelijk door boven methodes “@Async” te doen de methode Asynchroon laten werken. Dit zal er dan voor zorgen dat de flow van je applicatie niet wordt geblokkeerd

Er zijn dan zeker nog positieve punten die je kunt ophalen maar wij gaan nu over wat nadelen praten. Het eerste nadeel we gaan overlopen is dat het moeilijk te volgen kan zijn wat er precies in de applicatie gebeurt. Wanneer meerdere @EventListeners tegelijk reageren, kan het onduidelijk worden in welke volgorde alles wordt uitgevoerd, wat het debuggen bemoeilijkt.

Een tweede nadeel is dat er impliciete afhankelijkheden kunnen ontstaan tussen componenten. Listeners kunnen onbedoeld elkaar beïnvloeden of bijwerkingen veroorzaken, wat kan leiden tot onverwachte resultaten binnen de applicatie.